reaching the final answer. These states together provide evidence that the models "apply" the relevant functions (e.g., `get_capital`) abruptly at some mid-late layer to retrieve the answer. Moreover, in these cases, the model prepares the argument to this function in the layers prior to that in which the function is applied.

In Figure 2 we illustrate an example of the stages we observe across models. For the first several layers, we see no movement on the words of interest. Then, during **Argument Formation**, the model first represents the argument to the desired relation in the residual stream. This means that the top token in the vocabulary distribution at some intermediate layer(s) is the subject the question inquires about (e.g., the x, in `get_capital(`x`)`). During **Function Application** we find that the model abruptly switches from the argument to the output of the function (the y, in `get_capital(`x`) = `y). We find that function application is typically applied by the FFN update at that layer to the residual stream. This is done by adding the output vector $\vec{o}$ of the FFN to the residual stream representation, thus transforming it with an additive update. We study these $\vec{o}$ vectors in detail in Section 4. Finally, the model enters **Saturation**[3], where the model recognizes it has solved the next token, and ceases updating the token representation for the remaining layers.

The trend can be characterized by an X-shaped pattern of the argument and final output tokens when plotting the ranks of the argument(x) and output (y) tokens. We refer to this behavior as argument-function processing. Figure 3 shows that this same processing signature can be observed consistently across tasks and models. Moreover, it appears to become more prominent as the models increase in size. Interestingly, despite large differences in number of layers and overall size, models tend to undergo this process at similar points proportionally in the model.

4 Implementation of Context-Independent Functions in FFN Updates

The above results on processing signature suggest that the models "apply" a function about 2/3rds of the way through the network with the addition of an FFN update. Here, we investigate the mechanism via which that function is applied more closely. Specifically, focusing on GPT2-Medium[4], we show that we can force the encoded function to be applied to new arguments in new contexts by isolating the responsible FFN output vector and then dropping into a forward pass on a new input.

4.1 $\vec{o}$ Vector Interventions

Consider the example in Figure 2. At layer 18, the residual stream ($\vec{x_{18}}$) is in argument formation, and represents the " Poland" token. At the end of layer 19, a function is applied, transforming $\vec{x_{19}}$ into the answer token " Warsaw.

As discussed in the previous section, we can isolate the function application in this case to FFN 19; let $\tilde{x_{19}}$ represent the residual stream after the attention update, but before the FFN update at layer 19 (which still represents Poland). Recall that the update made by FFN 19 is written $\text{FFN}_{19}(\tilde{x_{19}}) = \vec{o_{19}}$ and $\vec{x_{19}} = \tilde{x_{19}} + \vec{o_{19}}$. We find that $\vec{o_{19}}$ will apply the `get_capital` function regardless of the content of $\tilde{x_{19}}$. For example, if we add $\vec{o_{19}}$ to some $\tilde{x}$ which represents the " China" token, it will transform into " Beijing". Thus we refer to $\vec{o_{19}}$ as $\vec{o_{city}}$ since it retrieves the capital cities of locations stored in the residual stream. We locate such $\vec{o}$ vectors in the uppercasing and past tense mapping tasks in the examples given in Section 2.2, which we refer to as $\vec{o_{upper}}$ and $\vec{o_{past}}$, respectively.[5]

We test whether these updates have the same effect, and thus implement the same function, as they do in the original contexts from which they were extracted. To do so, we replace entire FFN layers with these vectors and run new inputs through the intervened model.[6]

Data: We are interested in whether the captured o vectors can be applied in a novel context, in particular, to a context that is otherwise devoid of cues as to the function of interest. Thus, we synthesize a new dataset where each entry is a string of three

[3]Saturation events are described in Geva et al. (2022a) where detection of such events is used to "early-exit" out of the forward pass

[4]We focus on one model because manual analysis was required in order to determine how to perform the intervention. See Appendix for results on GPT-J and Section 7 for discussion.

[5]In Appendix A, we extend these results to GPT-J, for which the same procedure leads to strong effects on uppercasing, but smaller overall positive effects on capital cities and past tensing (see Section 7).

[6]Which FFNs to replace is a hyperparameter; we find that replacing layers 18-23 in GPT2-Medium leads to good results. It also appears necessary to replace multiple FFNs at a time. See additional experiments in Appendix E. It is likely that the $\vec{o}$ vectors are added over the course of several layers, consistent with the idea gradual updates from Jastrzebski et al. (2017).

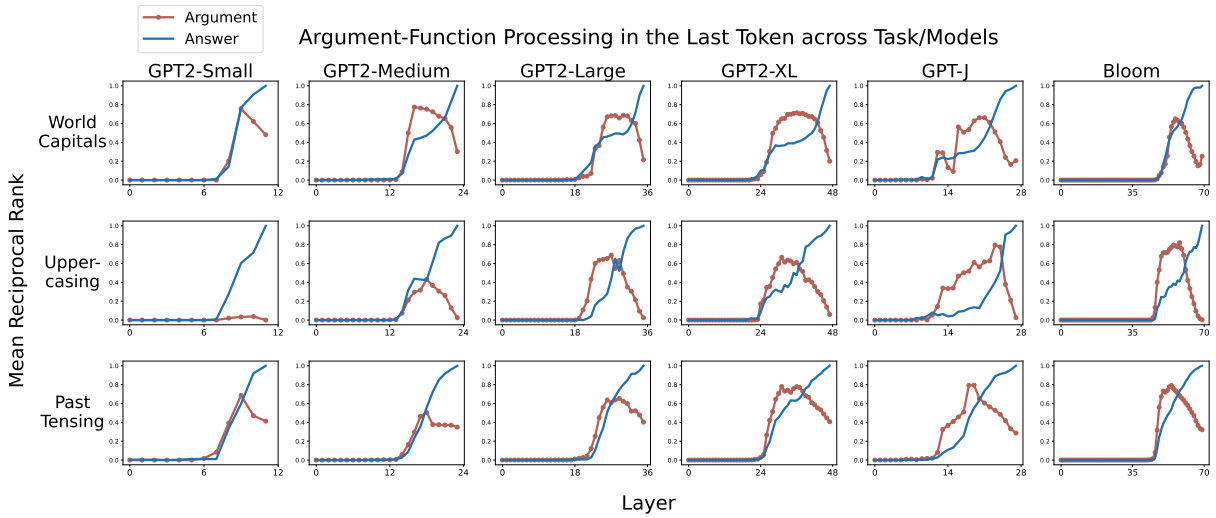

Figure 3: Argument formation and function application is characterized by a promotion of the argument (red) followed by it being replaced with the answer token (blue), forming an X when plotting reciprocal ranks. Across the three tasks we evaluate, we see that most of the models exhibit these traces, and despite the major differences in model depths, the stages occur at similar points in the models. Data shown is filtered by examples in which the models got the correct answer.

random tokens (with leading spaces) followed by a token x which represents a potential argument to the function of interest. For example, in experiments involving o_{city}, we might include a sequence such as `table mug free China table mug free China table mug free`. This input primes the model to produce "China" at the top of the residual stream, but provides no cues that the capital city is relevant, and thus allows us to isolate the effect of o_{city} in promoting "Beijing" in the residual stream. In addition to the original categories, we also include an "out-of-domain" dataset for each task: US states and capitals, 100 non-color words, and 128 irregular verbs. These additional data test the sensitivity of the $\vec{o}$ vectors to different types of arguments.

Results: Figure 4 shows results for a single example. Here, we see that "Beijing" is promoted all the way to the top of the distribution solely due to the injection of $\vec{o_{city}}$ into the forward pass. Figure 5 shows that this pattern holds in aggregate. In all settings, we see that the outputs of the intended functions are strongly promoted by adding the corresponding $\vec{o}$ vectors. By the last layer, for world and state capitals, the mean reciprocal rank of the target city name across all examples improves from roughly the 10th to the 3rd-highest ranked word and 17th and 4th-ranked words respectively. The target output token becomes the top token in 21.3%, 53.5%, and 7.8% of the time in the last layer in the world capitals, uppercasing, and past tensing tasks,

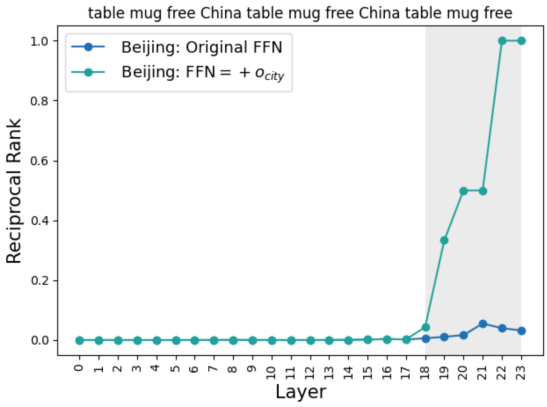

Figure 4: The gray area indicates layers with the FFN intervention. Even if the input context is nonsense (repeating pattern), when "China" is represented in the residual stream, the $\vec{o_{city}}$ vector promotes the correct capital city.

respectively.

We also see the promotion of the proper past tense verbs by $\vec{o_{past}}$. The reciprocal ranks improve similarly for both regular (approx. 7th to 3rd rank) and irregular verbs (approx. 6th to 3rd), indicating that the relationship between tenses is encoded similarly by the model for these two types. $\vec{o_{upper}}$ promotes the capitalized version of the test token almost every time, although the target word starts at a higher rank (on average, rank 5). These results together show that regardless of the surrounding context and the argument to which it is applied, $\vec{o}$ vectors consistently apply the expected functions. Since each vector was originally extracted from the model's processing of a single naturalistic in-

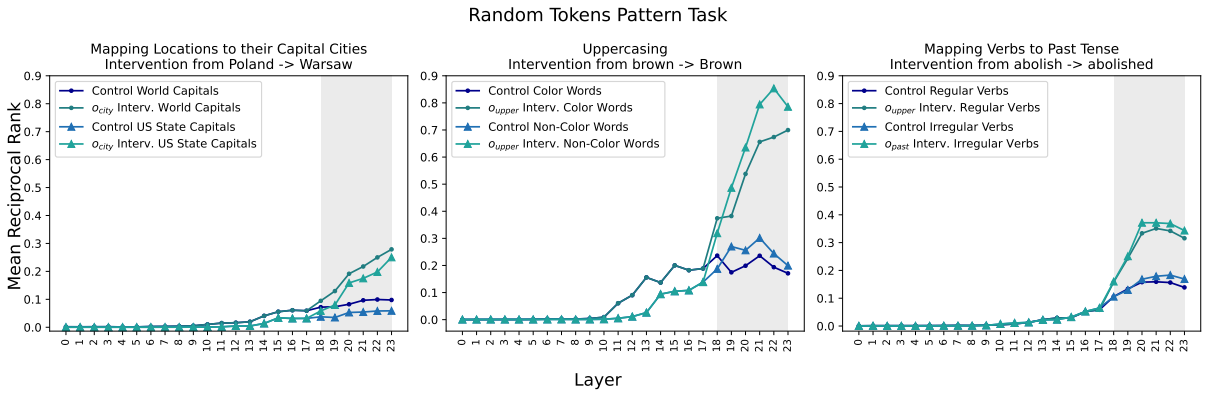

Figure 5: We intervene on GPT2-Medium's forward pass while it is predicting the completion of a pattern. The control indicates normal model execution, while the gray boxes indicate which FFNs are replaced with our selected $\vec{o}$ vectors. We can see a significant increase in the reciprocal rank of the output of the function implemented by the $\vec{o}$ vector used even though the context is completely absent of any indication of the original task.

put, this generalizability suggests **cross-context abstraction within the learned embedding space**.

Common Errors: While the above trend clearly holds on the aggregate, the intervention is not perfect for individual cases. The most common error is that the intervention has no real effect. In the in-domain (out-domain) settings, this occurred in about 37% (20%) of capital cities, 4% (5%) on uppercasing, and 19% (22%) for past tensing. We believe the rate is so much higher for world capitals because the model did not have a strong association between certain country-capital pairs from pretraining, e.g, for less frequently mentioned countries. Typically, in these cases, the top token remains the argument, but sometimes becomes some random other city, for example, predicting the capital of Armenia is Vienna. We also find that the way tokenization splits the argument and target words affects the ability of the $\vec{o}$ vector to work and is another source of errors. This is discussed further in Appendix F.

5 The Role of FFNs in Out-of-Context Retrieval

So far, we have shown that FFN output vectors can encode functions that transfer across contexts. Here, we investigate the role of this mechanism when we control whether the answer occurs in context. The tasks we study previously require recalling a token that does not appear in the given context (abstractive tasks). In this section we show that mid-higher layer FFNs are crucial for this process. When the answer to the question *does* appear in context (extractive tasks), we find that ablating a subset of FFNs has a comparatively minor effect on performance, indicating that they are relatively modular and there is a learned division of labor within the model. This observation holds across the decoder-only LMs tested in this paper. This breakdown is consistent with previous work finding that FFNs store facts learned from pre-training (Geva et al., 2021b; Meng et al., 2022b,c) and attention heads copy from the previous context (Wang et al.; Olsson et al., 2022).

5.1 Abstractive vs. Extractive Tasks

Extractive Tasks: Extractive tasks are those in which the exact tokens required to answer a prompt can be found in the input context. These tasks can thus be solved by parsing the local context alone, and thus do not necessarily require the model to apply a function of the type we have focused on in this paper (e.g., a function like `get_capital`).

Abstractive Tasks: Are those in which the answer to a prompt is not given in the input context and must be retrieved from pretraining memory. Our results suggest this is done primarily through argument-function processing, requiring function application through (typically) FFN updates as described in Section 3.

We provide examples with their associated GPT2-Medium layerwise decodings in Figure 7. We expect that the argument formation and function application stages of processing occur primarily in abstractive tasks. Indeed, in Appendix A, we show that the characteristic argument-answer X pattern disappears on extractive inputs. We hypothesize that applying out-of-context transformations